

Course Outline: Common Backend Architectures

Title: Common Backend Architectures **Learnpack:** + Arquitecturas más comunes en el backend **Prerequisite:** Python fundamentals (functions, lists, dictionaries) **Target Audience:** Beginner developers learning backend development **Total Lessons:** 8 **Estimated Total Length:** ~4,200 words **Format:** Conceptual (0% hands-on)

Course Description

This course introduces the foundational architectural patterns used in backend development. Students will learn how to recognize and understand MVC, layered architecture, and serverless approaches — the three most common ways to organize backend code.

Understanding architecture is not about memorizing diagrams. It's about developing the mental models to read existing codebases, communicate with other developers, and make informed decisions about how to structure applications. Students won't build these architectures from scratch yet; instead, they'll gain the vocabulary and conceptual foundation needed for the FastAPI work that follows.

By the end of this course, students will be able to identify architectural patterns in real projects, understand the trade-offs between different approaches, and recognize common anti-patterns that lead to unmaintainable code.

Course Philosophy

This course emphasizes:

- Recognition over implementation: Learn to identify patterns before building them
- Trade-offs over "best practices": Every architecture has strengths and weaknesses
- Vocabulary building: Speak the same language as professional developers
- Real-world context: Understand why these patterns exist and when to use them

Course Statistics Summary

Section	Lessons	Estimated Words
00 - Welcome	1	~450
01 - Model-View-Controller	2	~1,000
02 - Layered Architecture	2	~1,000
03 - Serverless Architecture	1	~500
04 - Assessment	1	~700
05 - Wrapping Up	1	~450
Total	8	~4,100

Section 00: Welcome

00.0 Why Architecture Matters 📖 (~450 words)

Learning Objectives:

- Explain what software architecture means in backend development

- Recognize that architecture decisions affect maintainability and scalability
- Understand why learning architecture patterns helps you read and contribute to existing codebases

Content Outline:

- What is software architecture? (The high-level organization of code)
- The building analogy: blueprints before construction
- Why backends need structure (multiple developers, changing requirements, growth)
- What you'll learn: three common patterns and how to recognize them

Transitions: This lesson sets the stage for exploring specific architectural patterns. Next, we'll dive into the most widely-known pattern: Model-View-Controller.

Key Principles:

- Architecture is about organization, not complexity
- Good architecture makes code easier to change, not harder
- You don't need to memorize — you need to recognize

Examples:

- A messy room vs. an organized closet: both hold clothes, but one is easier to find things in
 - A restaurant kitchen: stations for prep, cooking, and plating exist for a reason
-

Section 01: Model-View-Controller (MVC)

01.0 Understanding MVC 📖 (~500 words)

Learning Objectives:

- Define the three components of MVC: Model, View, and Controller
- Explain the responsibility of each component
- Identify MVC structure when looking at a codebase

Content Outline:

- The origin of MVC (separating concerns in user interfaces)
- Model: the data and business logic (what your app knows)
- View: the presentation layer (what users see)
- Controller: the coordinator (handles requests, connects Model and View)
- Why separation matters: change one part without breaking others

Transitions: Now that you know what each component does, let's see how they work together when a user makes a request.

Key Principles:

- Each component has one job (single responsibility)
- Components communicate through well-defined interfaces
- The Model knows nothing about how data is displayed

Examples:

- A restaurant: the kitchen (Model) prepares food, the dining room (View) presents it, the waiter (Controller) coordinates orders
 - An e-commerce site: product data (Model), product page HTML (View), route handler (Controller)
-

01.1 How Data Flows in MVC 📖 (~500 words)

Learning Objectives:

- Trace a user request through the MVC cycle
- Explain why the Controller sits between Model and View
- Recognize the request-response flow in backend applications

Content Outline:

- The lifecycle of a request: user action → Controller → Model → View → response
- Step-by-step walkthrough: "Show me product #42"
- Why the View never talks directly to the Model
- MVC in APIs: the "View" becomes JSON responses
- Common variations: MVP, MVVM (brief mention, not deep dive)

Transitions: MVC is a great starting point, but as applications grow, developers often need more structure. Next, we'll explore layered architecture, which adds more separation between concerns.

Key Principles:

- Data flows in one direction through the system
- The Controller is the entry point for all external requests
- In APIs, the "View" is often just data serialization (JSON)

Examples:

- User clicks "Add to Cart" → Controller receives request → Model updates cart data → View returns updated cart JSON
 - Diagram: Request → Controller → Model ↔ Database → Controller → View → Response
-

Section 02: Layered Architecture

02.0 The Layered Approach 📖 (~500 words)

Learning Objectives:

- Describe the concept of layers in software architecture
- Identify common layers: presentation, business logic, data access
- Explain the rule that layers only communicate with adjacent layers

Content Outline:

- What is layered architecture? (Horizontal slices of responsibility)
- The three classic layers: Presentation, Business, Data
- The "onion" rule: outer layers depend on inner layers, never the reverse
- Benefits: testability, replaceability, team organization
- Drawbacks: can become rigid, potential for too many layers

Transitions: Layered architecture provides clear boundaries, but some developers wanted even more flexibility. That led to hexagonal architecture, which we'll explore next.

Key Principles:

- Each layer has a specific responsibility
- Dependencies flow inward (toward the core business logic)
- Layers can be replaced independently (swap databases, change UI)

Examples:

- A three-tier web app: React frontend (Presentation) → Python API (Business) → PostgreSQL (Data)
 - Changing from MySQL to PostgreSQL only affects the Data layer
-

02.1 Introduction to Hexagonal Architecture 📖 (~500 words)

Learning Objectives:

- Explain the core idea of hexagonal architecture (ports and adapters)
- Understand why business logic should be isolated from external systems
- Recognize when hexagonal architecture might be overkill

Content Outline:

- The problem hexagonal solves: business logic tangled with frameworks and databases
- Core concept: the application is a hexagon with "ports" (interfaces) on each side
- Adapters: implementations that plug into ports (database adapter, HTTP adapter)
- The benefit: your business logic doesn't know or care about external systems
- When to use it: complex domains, long-lived applications
- When to skip it: simple CRUD apps, prototypes, learning projects

Transitions: We've covered architectures where you manage your own servers and code structure. But what if you didn't need servers at all? That's the promise of serverless architecture.

Key Principles:

- Business logic is the core; everything else is an adapter
- External systems (databases, APIs, frameworks) are details, not foundations
- Hexagonal is about flexibility, not complexity for its own sake

Examples:

- A payment system: the core knows "process payment," but whether it uses Stripe or PayPal is an adapter detail
 - Testing: swap the database adapter for an in-memory adapter during tests
-

Section 03: Serverless Architecture

03.0 What Is Serverless? 📖 (~500 words)

Learning Objectives:

- Define serverless architecture and how it differs from traditional backends
- List the main benefits and drawbacks of serverless
- Identify use cases where serverless makes sense

Content Outline:

- The name is misleading: there are servers, you just don't manage them
- How it works: functions triggered by events (HTTP requests, file uploads, timers)
- Pay-per-execution: you pay only when code runs
- Major providers: AWS Lambda, Google Cloud Functions, Vercel Functions
- Benefits: no server maintenance, automatic scaling, cost efficiency for sporadic loads
- Drawbacks: cold starts, vendor lock-in, harder debugging, stateless constraints
- When to use: event-driven tasks, APIs with variable traffic, background jobs
- When to avoid: long-running processes, real-time applications, complex stateful logic

Transitions: You've now seen three major architectural approaches. But knowing the patterns isn't enough — you also need to recognize when things go wrong. Let's look at common anti-patterns before testing your knowledge.

Key Principles:

- Serverless trades control for convenience
- Functions should be small, focused, and stateless

- Not every application benefits from serverless

Examples:

- Image resizing: user uploads photo → Lambda function creates thumbnails → stores in S3
 - A startup with unpredictable traffic: serverless scales automatically without paying for idle servers
-

Section 04: Assessment

04.0 Backend Architecture Knowledge Check 🧠 (~700 words)

Learning Objectives:

- Demonstrate understanding of MVC, layered, and serverless architectures
- Apply architectural concepts to realistic scenarios
- Identify anti-patterns and explain why they're problematic

Question Format: Scenario-based (5 questions)

Questions:

Question 1: A junior developer puts database queries directly inside the route handlers of a FastAPI application. When the team wants to switch from PostgreSQL to MongoDB, they realize they need to modify every single route.

Which architectural principle was violated, and which pattern would have prevented this problem?

A) Single responsibility was violated; MVC would have helped by separating the Controller from data access B) The layered approach was ignored; a separate Data layer would isolate database logic C) Serverless should have been used to avoid database concerns entirely D) Hexagonal architecture was needed; ports and adapters would have made the database a replaceable detail

Correct: B (also accept D as valid — both address the issue)

Question 2: You're reviewing a codebase where the Model directly sends emails to users whenever data changes. The View also makes database calls to fetch additional information.

What's wrong with this design?

A) Nothing — this is standard MVC practice B) The Model should not have side effects like sending emails; the View should not access the database directly C) The Controller is missing entirely D) This should be serverless instead

Correct: B

Question 3: A company builds an image processing API. Traffic is highly unpredictable — sometimes zero requests per hour, sometimes thousands. Each image processing task takes about 2 seconds. They want to minimize costs.

Which architecture best fits this scenario?

A) Traditional MVC with a dedicated server B) Layered architecture with horizontal scaling C) Serverless functions triggered by image upload events D) Hexagonal architecture with multiple adapters

Correct: C

Question 4: In hexagonal architecture, what is the purpose of an "adapter"?

A) To convert data between different formats like JSON and XML B) To implement a specific technology (database, API, UI) that connects to a port C) To speed up communication between layers D) To replace the Controller in MVC

Correct: B

Question 5: A developer argues: "We should use hexagonal architecture for our weekend hackathon project — it's the most modern approach."

What's the best response?

A) Agree — hexagonal is always the best choice B) Disagree — hexagonal adds complexity that doesn't benefit a simple, short-lived project C) Suggest serverless instead since it's newer D) Recommend MVC because it's older and more reliable

Correct: B

Evaluation Criteria:

- 5/5: Excellent understanding of architectural patterns and trade-offs
 - 4/5: Solid grasp with minor gaps
 - 3/5: Basic understanding; review weak areas
 - 2/5 or below: Re-read the course and focus on the examples
-

Section 05: Wrapping Up

05.0 Your Architecture Journey (~450 words)

Content Outline:

You've just completed your first tour of backend architecture patterns. Let's recap what you've learned and where this knowledge takes you next.

You now understand three fundamental approaches to organizing backend code. MVC separates your application into Model (data and logic), View (presentation), and Controller (coordination). Layered architecture stacks responsibilities into horizontal slices — presentation, business logic, and data access. Serverless shifts the model entirely, letting you write functions that run on-demand without managing infrastructure.

More importantly, you've learned that architecture is about trade-offs, not "best practices." MVC is simple and widely understood but can become tangled in large applications. Layered architecture provides clear boundaries but can feel rigid. Hexagonal architecture offers maximum flexibility but adds complexity you might not need. Serverless eliminates infrastructure headaches but introduces cold starts and vendor lock-in.

The goal was never to memorize definitions. It was to build the mental models that let you look at a codebase and understand how it's organized. When you start working with FastAPI in the upcoming lessons, you'll recognize these patterns in action. You'll see how routes act as Controllers, how Pydantic models represent your data layer, and how separating concerns makes your code easier to test and modify.

Architecture knowledge also helps you communicate. When a senior developer mentions "moving to a more hexagonal approach" or asks you to "keep business logic out of the Controller," you'll know what they mean. This shared vocabulary is essential for working on teams.

What's Next:

The next lessons will introduce FastAPI, where you'll start building actual backend applications. You'll see MVC principles applied: routes handle requests (Controller), Pydantic schemas define data shapes (Model), and JSON responses serve as your View. Later, as you work with databases and authentication, layered thinking will help you keep your code organized.

Don't worry about choosing the "perfect" architecture right now. Start simple. Apply what you've learned. Recognize patterns when you see them. Architecture understanding deepens with experience — every codebase you read, every refactor you attempt, every "why is this so hard to change?" moment teaches you something new.

Resources for Further Exploration:

- Martin Fowler's website (martinfowler.com) — deep dives on architectural patterns
- "Clean Architecture" by Robert C. Martin — hexagonal architecture explained
- AWS Lambda documentation — serverless concepts and tutorials

You're ready to build. Let's move on to FastAPI.

